

Software Requirements Specification for BT-Mux (Virtual Socket Multiplexer)

Version: 1.0 Approved

Prepared by: Technical Lead

Organization: Indian Institute of Information Technology (IIIT), Pune

Date Created: August 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for **BT-Mux (v1.0)**, an application-agnostic, user-space virtual socket middleware library and experimental proof-of-concept system. BT-Mux is designed to bypass physical connection constraints on standard Bluetooth Low Energy (BLE) hardware controllers by multiplexing multiple logical streams over a restricted set of physical radio links.

1.2 Document Conventions

This document follows the Karl Wiegers SRS template guidelines. Requirement priorities are classified as **High**, **Medium**, or **Low**. Child requirements inherit the priority rating of their parent system feature. Standard requirement identifiers follow the tag format **REQ-X**. Mathematical variables include N (total logical peripheral nodes), N_{max} (maximum concurrent physical radio links), and T_{slice} (TDM duty-cycling time-window duration).

1.3 Intended Audience and Reading Suggestions

This document is intended for project evaluators, academic advisors, systems developers, testbed leads, and documentation authors.

- **Academic Advisors / Evaluators:** Focus on Sections 1.4 (Scope), 2.1 (Product Perspective), 3 (System Features), and 5 (Nonfunctional Requirements).
-
- **Core Engine Developers:** Focus on Sections 2.4, 2.5, 3.1, 3.2, and 4.3.
-
- **Testbed & UI Developers:** Focus on Sections 3.3, 3.4, 4.1, and 4.4.

1.4 Project Scope

BT-Mux is a software-defined Virtual Socket Multiplexing Middleware. Its primary objective is to enable a central host device to maintain $N \geq 10$ concurrent, persistent logical data streams over a physical Bluetooth radio limited to $N_{\text{max}} \approx 4$ active GATT connections. It accomplishes this using time-division duty cycling (TDM), local session state

preservation in RAM, and RSSI-guided scheduling without requiring low-level controller firmware modifications.

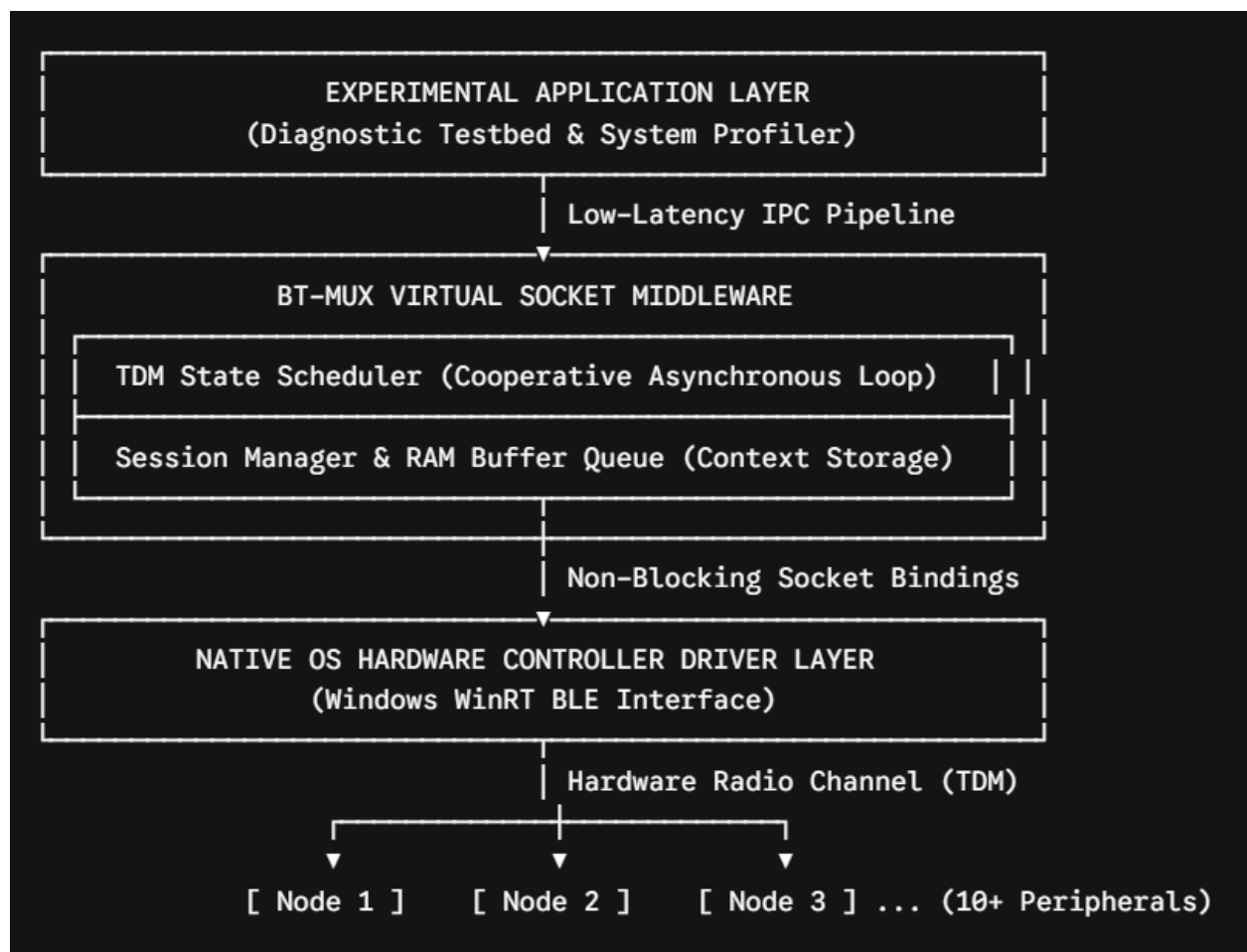
1.5 References

- IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
-
- Bluetooth SIG: *Bluetooth Core Specification v5.3 (GATT/GAP Layers)*.
-
- Karl E. Wieggers: *Software Requirements Specification Template*.
-

2. Overall Description

2.1 Product Perspective

BT-Mux is a standalone user-space middleware system that sits directly between high-level application sockets and native operating system Bluetooth drivers (Windows WinRT BLE API).



2.2 Product Features

- **Virtual Socket Abstraction:** Presents persistent, non-blocking socket interfaces to higher-level applications, masking underlying physical connection drops.
-
- **TDM Round-Robin Scheduler:** Rotates active GATT physical radio slots across peripheral batches using non-blocking asynchronous event loops.
-
- **Session Context & Buffer Manager:** Caches unacknowledged packet byte streams locally in memory prior to physical disconnects, flushing queues upon physical link restoration.
-
- **Real-Time Diagnostic Profiler:** Provides a live visual network topology graph, tracking active vs. paused nodes, context-switch handoff latency, and packet delivery metrics.
-
- **Experimental Telemetry Store:** Persists quantitative benchmark logs (Packet Delivery Ratio, Round-Trip Latency, RSSI) for empirical evaluation.
-

2.3 User Classes and Characteristics

- **System Researcher / Evaluator:** Configures TDM parameters (e.g., T_{slice} durations), executes benchmark scripts, and analyzes network performance metrics.
-
- **Application Developer:** Imports the BT-Mux library interface to send and receive data payloads without managing physical BLE connection lifecycles.
-
- **Peripheral Edge Node (Automated Device):** A remote sensor node or simulated BLE GATT server responding to central host polling requests.
-

2.4 Operating Environment

- **Central Host Hardware:** x86_64 architecture laptop/PC equipped with an internal or USB Bluetooth 4.2+ Low Energy controller chip.
-
- **Central Host Operating System:** Windows 10/11 (Version 2004 or higher) supporting the native WinRT Bluetooth API stack.
-
- **Runtime Environment:** Asynchronous V8/C++ user-space runtime environment.
-
- **Peripheral Hardware:** 10+ BLE-enabled devices (secondary test phones, ESP32 microcontrollers, or simulated GATT server processes).
-

2.5 Design and Implementation Constraints

- **Hardware Connection Limit (N_{max}):** Enforced by physical host radio registers, restricting simultaneous physical GATT links to 4–7.
-

- **Connection Re-establishment Latency:** Bluetooth GAP handshake and service discovery incur a fixed latency penalty of 100 ms to 300 ms per physical slot reconnection.
-
- **User-Space Operation:** System must operate purely in user-space without modifying native Windows kernel drivers or Bluetooth controller firmware.
-

2.6 User Documentation

Deliverables include the BTP Engineering Thesis Report, System Architecture Specifications, API Reference Manual, and empirical Benchmark Logs.

2.7 Assumptions and Dependencies

- **PoC Target Scale:** The proof-of-concept explicitly targets $N \geq 10$ peripheral nodes over $N_{\text{max}} = 4$ physical slots, with zero protocol-level upper bounds ($N \rightarrow \infty$).
-
- **GATT Pairing Configuration:** Peripheral devices are assumed to operate in non-bonded or auto-accepted GATT configurations to eliminate manual passkey prompts during rapid reconnects.
-

3. System Features

3.1 TDM Connection Scheduler

- **3.1.1 Description and Priority:** High Priority. Rotates physical radio link assignments across peripheral node batches using non-blocking asynchronous timing queues.
-
- **3.1.2 Stimulus/Response Sequences:**
- - Application initializes N virtual socket streams.
 -
 - Scheduler fetches the current active batch of size N_{max} .
 -
 - System establishes physical links, processes buffered data, cleanly disconnects, and advances to the next batch upon timer expiration (T_{slice}).
 -
- **3.1.3 Functional Requirements:**
- - **REQ-1:** The system shall dynamically detect the host controller's physical connection cap N_{max} .
 -

- **REQ-2:** The TDM scheduler shall execute time-sliced batch switching across target nodes using configurable time windows ($100 \text{ ms} \leq T_{\text{slice}} \leq 2000 \text{ ms}$).
-
- **REQ-3:** The system shall issue graceful GATT disconnect calls to freeing radio slots prior to connecting the next node batch.
-
- **REQ-4:** If a target peripheral fails to establish a physical link within a timeout threshold (1000 ms), the scheduler shall flag the node as **DEGRADED**, bypass its current slot, and proceed to the next queued node.
-

3.2 Session Context & RAM Buffer Manager

- **3.2.1 Description and Priority:** High Priority. Preserves logical state and queues outgoing/incoming byte streams while physical links are disconnected.
-
- **3.2.2 Stimulus/Response Sequences:**
- - Higher-level app writes payload to a physically disconnected node's virtual socket.
 -
 - Session Manager intercepts payload and appends it to an in-memory queue.
 -
 - When TDM scheduler establishes physical connection to target node, Session Manager flushes queue across GATT characteristic.
 -
- **3.2.3 Functional Requirements:**
- - **REQ-5:** The system shall provide non-blocking Virtual Socket abstractions that remain in an **ACTIVE** state regardless of underlying physical radio link status.
 -
 - **REQ-6:** The system shall maintain dedicated, thread-safe RAM FIFO buffer queues for each logical stream.
 -
 - **REQ-7:** Outgoing payloads shall be cached in RAM during physical disconnect phases without throwing socket drop errors to the application.
 -
 - **REQ-8:** The system shall automatically flush pending queue bytes immediately upon receiving a physical **GATT_CONNECTED** event.

3.3 Real-Time Diagnostic Profiler & Network Testbed

- **3.3.1 Description and Priority:** Medium Priority. Visualizes real-time multiplexer state transitions, node topologies, and context-switch latencies.
-
- **3.3.2 Stimulus/Response Sequences:**
- - TDM scheduler executes a state transition.
 -
 - Middleware emits state event payload over low-latency IPC pipeline.
 -
 - Diagnostic profiler UI updates visual topological graph.
 -
- **3.3.3 Functional Requirements:**
- - **REQ-9:** The system shall provide an interactive visual canvas rendering all N logical nodes and their current states (**ACTIVE_PHYSICAL**, **PAUSED_QUEUED**, **DISCONNECTED**, **ERROR**).
 -
 - **REQ-10:** The profiler shall display real-time metrics including active physical slot utilization, average context-switch handoff latency, and packet queue depths.
 -
 - **REQ-11:** The profiler interface shall provide controls to dynamically adjust T_{slice} durations during active test runs.
 -

3.4 Experimental Telemetry Logger

- **3.4.1 Description and Priority:** Low Priority. Records system benchmarking metrics for quantitative evaluation in the thesis report.
-
- **3.4.2 Functional Requirements:**
- - **REQ-12:** The system shall record timestamped logs of packet transmission success, round-trip times (RTT), and RSSI values to a local database.
 -
 - **REQ-13:** The system shall export benchmark session summaries to CSV format, computing global Packet Delivery Ratios (PDR) and average handoff overhead.

4. External Interface Requirements

4.1 User Interfaces

The system provides a Web-based Diagnostic Profiler UI rendered in standard modern web browsers.

- **Topology Canvas:** Interactive node graph displaying active radio connections vs. paused logical queues.
-
- **Control Panel:** Input controls for setting T_{slice} intervals, manual node force-connects, and session log controls.
-
- **Telemetry Panels:** Live updating line graphs for throughput, latency, and packet loss percentages.

DIAGNOSTIC PROFILER	
TOPOLOGY GRAPH	LIVE TELEMETRY METRICS
[Node A] — (Active Slot 1)	TDM Slice (T_{slice}): 500ms
[Node B] — (Active Slot 2)	Active Slots: 3 / 3
[Node C] — (Active Slot 3)	Total Streams: 12 Logical
[Node D] .. (Queued RAM)	Avg Handoff Latency: 185ms
[Node E] .. (Queued RAM)	Packet Delivery Ratio: 97.4%
[Node F] .. (Queued RAM)	

4.2 Hardware Interfaces

- **Host Bluetooth Controller:** Interfaces with x86_64 PC internal/USB Bluetooth hardware radios via standard host controller interface protocols.
-

4.3 Software Interfaces

- **Native Operating System Driver API:** Interfaces with Windows WinRT Bluetooth APIs ([Windows.Devices.Bluetooth.GenericAttributeProfile](#)) via user-space asynchronous bindings ([@abandonware/noble](#)).
-

- **Database Driver:** Local database connectivity via standard asynchronous storage drivers.

-

4.4 Communications Interfaces

- **Bluetooth GATT Profile:** Communications over standard Generic Attribute Profile (GATT) custom characteristics using UUID-based read/write channels.
-
- **Inter-Process Communication (IPC):** Low-latency WebSocket protocol streaming JSON-formatted system state events between the middleware engine and the visual profiler.

-

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **Context-Switch Handoff Latency:** Physical link release and re-establishment for a slot swap shall complete within $\leq 300 \text{ ms}$ under normal radio conditions.
-
- **Packet Delivery Ratio (\$PDR\$):** The system shall achieve a global \$PDR \geq 95\%\$ across \$N = 10\$ logical streams during baseline testing.
-
- **Memory Footprint:** The RAM buffer queue engine shall maintain a baseline footprint under 150 MB on the host system.

-

5.2 Safety Requirements

The software operates purely in user-space and controls short-range wireless radios; no physical hazard or safety certification requirements apply.

5.3 Security Requirements

The system shall bypass OS passkey prompts by utilizing non-authenticated custom GATT channels or pre-paired device profiles to prevent user interaction bottlenecks during rapid TDM swapping.

5.4 Software Quality Attributes

- **Robustness:** If a peripheral node unexpectedly drops out of physical radio range, the scheduler shall flag the link as invalid within 1 cycle and continue multiplexing remaining nodes without crashing.
-
- **Maintainability:** Modular architecture separating the TDM core, buffer queues, API routes, and diagnostic profiler.
-
- **Reusability:** BT-Mux core engine designed as a generic, application-agnostic middleware library.

6. Other Requirements

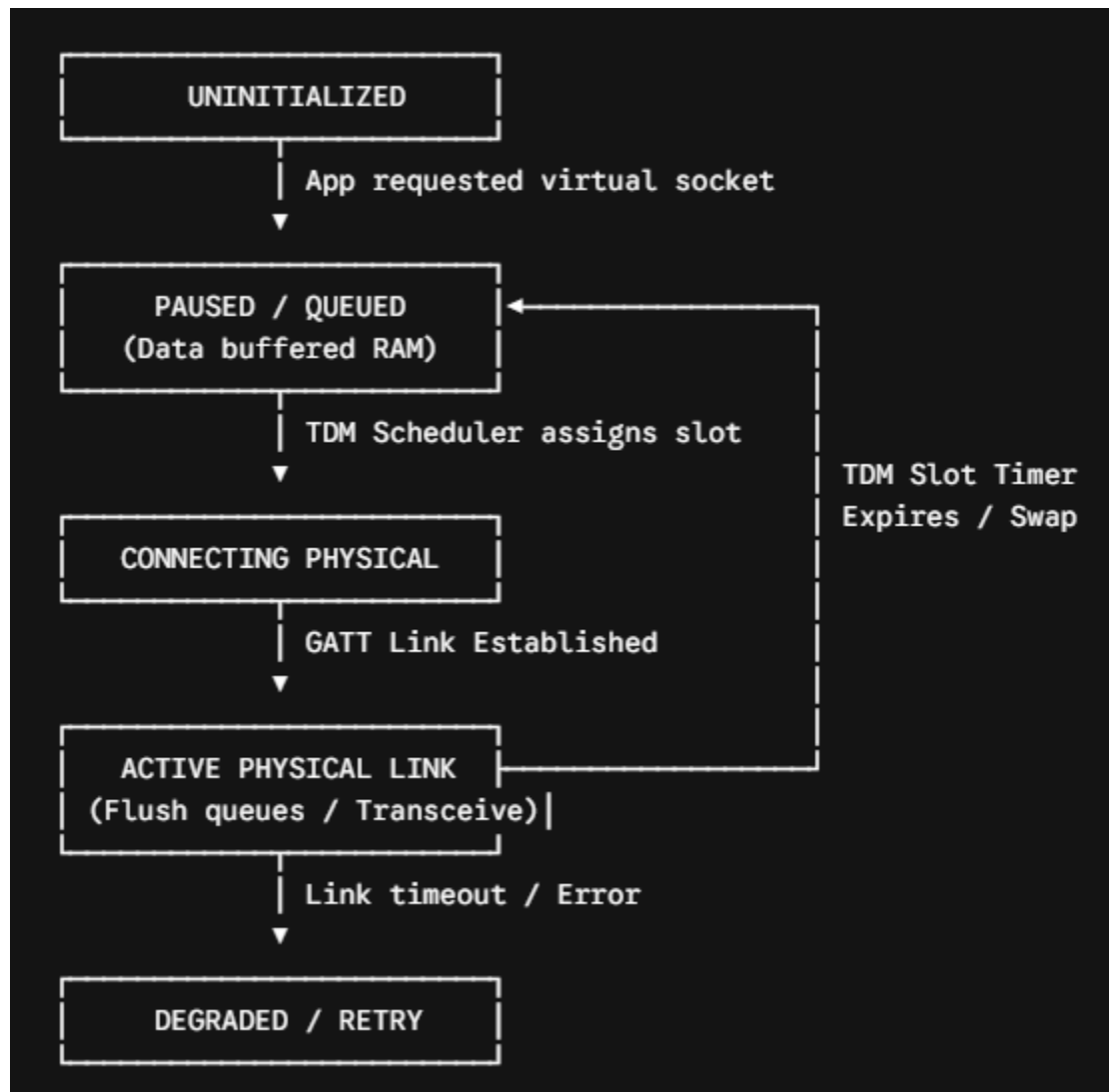
There are no additional regulatory, legal, or internationalization requirements applicable to this proof-of-concept system.

Appendix A: Glossary

- **BLE:** Bluetooth Low Energy.
-
- **GATT:** Generic Attribute Profile.
-
- **GAP:** Generic Access Profile.
-
- **TDM:** Time-Division Multiplexing.
-
- **IPC:** Inter-Process Communication.
-
- **PDR:** Packet Delivery Ratio ($\frac{\text{Total Packets Received}}{\text{Total Packets Sent}} \times 100\%$).
-
- **\$N\$:** Total number of logical peripheral streams requested by higher-level applications.
-
- **N_{max} :** Physical hardware limit of concurrent active Bluetooth GATT links enforced by the host radio controller.
-
- **T_{slice} :** Active physical time-slot duration allocated to a batch of nodes before executing a TDM context switch.

Appendix B: Analysis Models

State Machine Lifecycle of a BT-Mux Logical Socket



Appendix C: Issues List

1. **GATT Re-connection Latency Optimizations:** Evaluating connection parameter update requests to minimize fixed GAP handoff latency on Windows.
- 2.

3. **Dynamic Slot Scaling:** Testing RSSI-weighted TDM allocation heuristics (allocating larger T_{slice} windows to nodes with higher throughput demands or weaker signal strength) vs. pure round-robin.